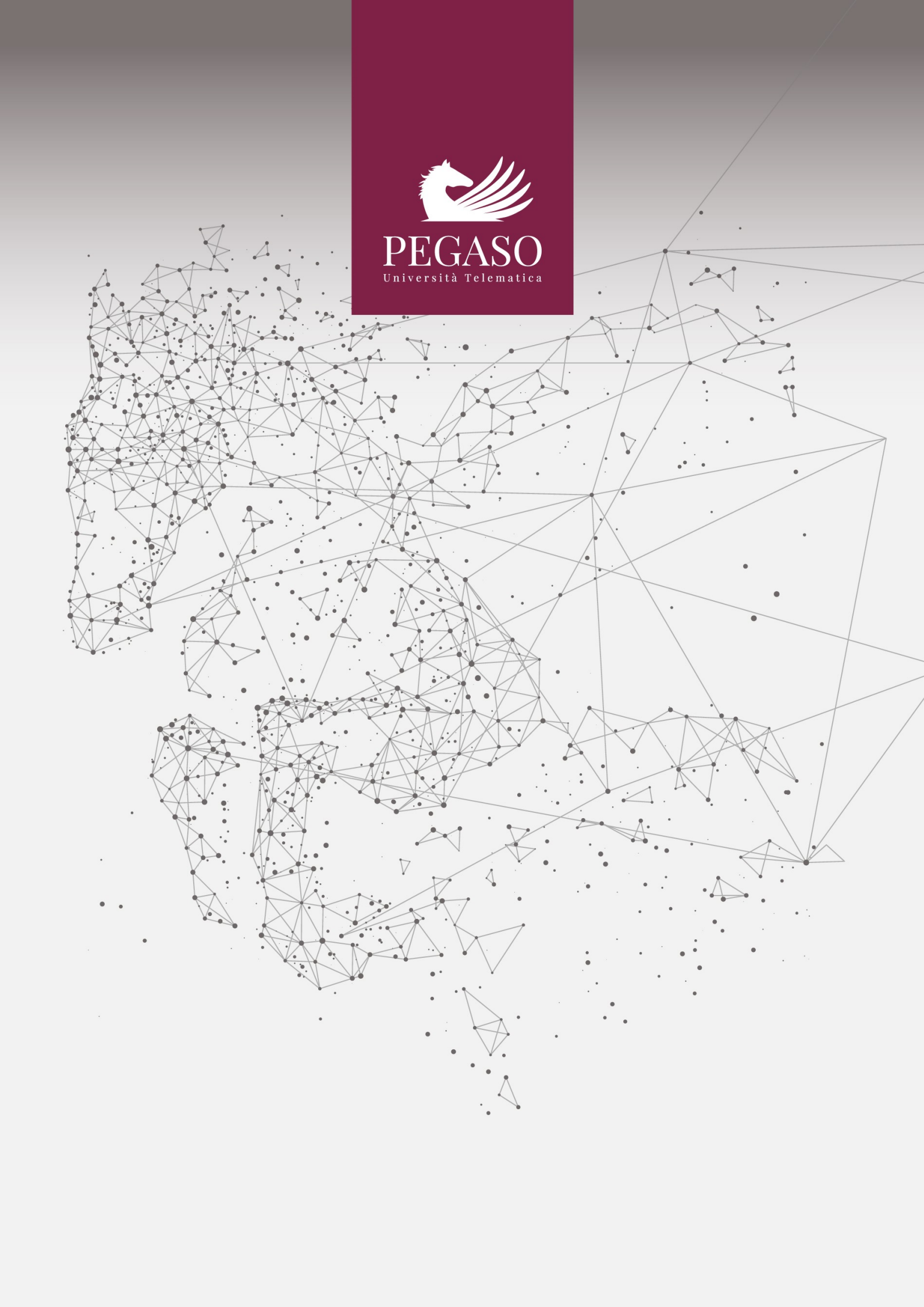

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. MVC....3
2. CONTROLLER....5
3. MODEL....10
BIBLIOGRAFIA....13
SITOGRAFIA....14

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. MVC

Lor MVC (Model-View-Controller) è un pattern architetturale utilizzato per separare le responsabilità del codice in tre parti distinte: il **Model**, la **View** e il **Controller**.

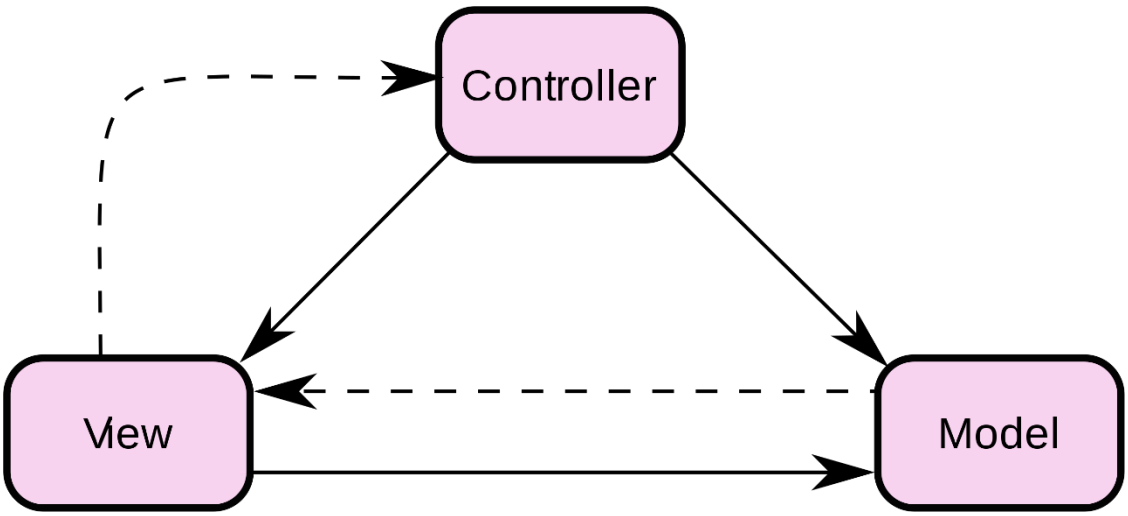


Figura 1 *- MVC*

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. Il Model non conosce nulla della View o del Controller, ma fornisce solo un'interfaccia per accedere ai dati.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. La View non ha alcuna conoscenza del Model o del Controller, ma fornisce solo un'interfaccia utente per interagire con l'applicazione.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. Il Controller riceve le richieste dell'utente dalla View, accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

L'utilizzo del pattern MVC ha diversi vantaggi, tra cui una maggiore separazione delle responsabilità del codice, una maggiore modularità, una maggiore flessibilità e una maggiore facilità di manutenzione e testabilità.

In Node.js, il pattern MVC può essere implementato utilizzando un framework web come Express.js. Ad esempio, si può utilizzare Express per gestire le richieste dell'utente (Controller), definire rotte e middleware per l'accesso ai dati dell'applicazione (Model) e utilizzare template engine come Handlebars o EJS per creare le View.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vediamo nel dettaglio i singoli componenti del pattern MVC e come possono essere implementati in Node.js.

- Il **Model** rappresenta i dati dell'applicazione e definisce le operazioni per accedere e manipolare questi dati. In Node.js, il Model può essere implementato utilizzando una libreria di accesso ai database come Sequelize, Mongoose o Knex.js. Ad esempio, Sequelize è un ORM (Object-Relational Mapping) che permette di definire i modelli dei dati dell'applicazione e di interagire con il database utilizzando oggetti JavaScript.
- La **View** rappresenta l'interfaccia utente dell'applicazione e visualizza i dati forniti dal Model. In Node.js, la View può essere implementata utilizzando un template engine come Handlebars, EJS o Pug. Ad esempio, Handlebars permette di creare template HTML dinamici utilizzando una sintassi simile a quella di Mustache.
- Il **Controller** si occupa di coordinare l'interazione tra il Model e la View. In Node.js, il Controller può essere implementato utilizzando le funzionalità di routing di Express.js. Ad esempio, si può utilizzare Express per definire le rotte dell'applicazione e le funzioni che gestiscono le richieste HTTP. In questo modo, il Controller accede ai dati del Model per elaborare la richiesta e aggiorna la View con i dati elaborati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Controller

Il controller è il componente del pattern MVC che gestisce la logica di business dell'applicazione. In una applicazione Node.js con Express, il controller può essere implementato come una funzione o come un modulo.

In generale, il controller si occupa di elaborare le richieste provenienti dal client, accedere ai dati necessari per elaborare la richiesta e poi restituire la risposta al client. La logica di business, come l'elaborazione dei dati, la validazione delle richieste e la gestione degli errori, dovrebbe essere gestita dal controller.

Ad esempio, se l'applicazione prevede la gestione di una lista di utenti, il controller dovrebbe gestire le richieste per visualizzare, creare, modificare ed eliminare gli utenti. Il controller dovrebbe accedere al modello per recuperare o modificare i dati degli utenti e poi restituire la risposta al client.

La separazione della logica di business dal resto dell'applicazione aiuta a creare un codice più modulare e mantenibile. Inoltre, permette di utilizzare gli stessi controller per diverse view o interfacce utente.

Ecco un esempio di come creare un controller come un modulo in un'applicazione Node.js con Express:

```
// file usersController.js
function listUsers(req, res) {
  const users = [
    { id: 1, name: 'John Doe' },
    { id: 2, name: 'Jane Smith' },
    { id: 3, name: 'Bob Johnson' },
  ];
  const filteredUsers = filterUsers(users, req.query.search);
  res.render('users', { users: filteredUsers });
}

function filterUsers(users, search) {
  if (!search) {
    return users;
  }
  return users.filter(user => {
    return user.name.toLowerCase().includes(search.toLowerCase());
  });
}
module.exports = {
  listUsers,
};
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, viene creato un modulo usersController che contiene una funzione listUsers() che gestisce la logica di business per visualizzare la lista degli utenti. La funzione filterUsers() è definita come una funzione privata del modulo.

Per utilizzare il modulo usersController nel file app.js, si deve richiamare il modulo e utilizzare la funzione listUsers() come callback per l'endpoint /users, ad esempio:

```
const express = require('express');
const app = express();

const usersController = require('./controllers/usersController');

app.get('/users', usersController.listUsers);

app.set('views', './views');
app.set('view engine', 'ejs');

app.listen(3000, () => {
  console.log('Server started on port 3000');
});
```

In questo modo, il controller gestisce la logica di business e la view si occupa solo di visualizzare i dati restituiti dal controller.

Il corrispondente file users.ejs sarà:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Users List</title>
</head>
<body>
  <h1>Users List</h1>
  <ul>
    <% users.forEach(user => { %>
      <li><%= user.name %></li>
    <% }); %>
  </ul>
</body>
</html>
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Richiamando il controller ottentiamo:

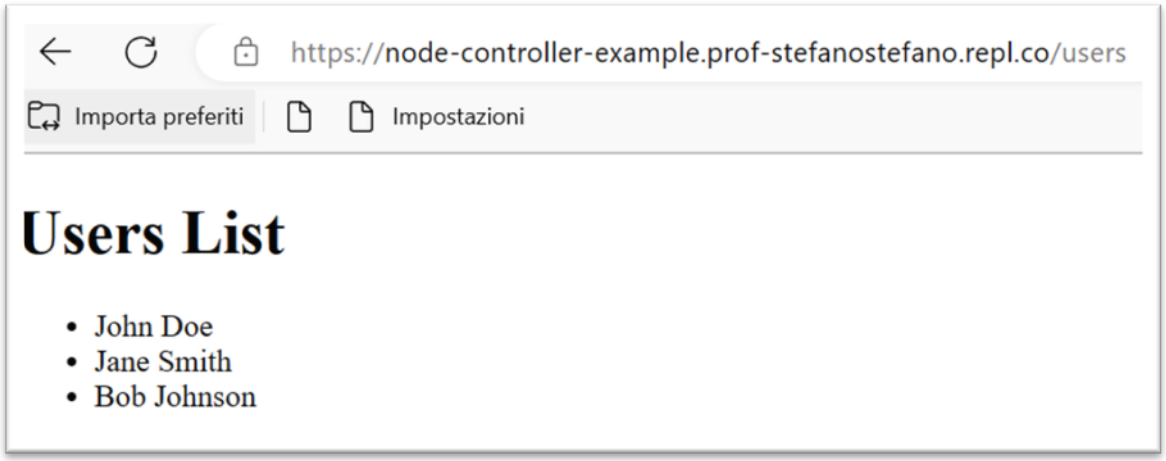


***Figura 2** - controller*

Il controller e la route sono due concetti distinti, ma correlati: la route è l'endpoint dell'applicazione, ovvero l'indirizzo URL che il client richiama per accedere alle risorse o alle funzionalità dell'applicazione. La route viene definita in una delle funzioni di routing di Express, come app.get(), app.post(), ecc.

Nel nostro esempio, la route /users viene definita come:

```
app.get('/users', usersController.listUsers);
```

In questo caso, la route /users viene associata alla funzione listUsers() del controller usersController.

Il controller, invece, è il componente che si occupa di gestire la logica di business dell'applicazione. In generale, il controller accetta le richieste dal client, accede ai dati necessari per elaborare la richiesta e restituisce una risposta al client. Nel nostro esempio, il controller usersController gestisce la richiesta della route /users utilizzando la funzione listUsers(), che filtra e restituisce una lista di utenti.

In sintesi, la route rappresenta l'indirizzo URL a cui il client accede per accedere alle risorse o alle funzionalità dell'applicazione, mentre il controller rappresenta la logica di business dell'applicazione che gestisce la richiesta del client e restituisce una risposta appropriata. In Express, la route e il controller sono correlati, poiché la route viene definita per associare una richiesta HTTP a una specifica funzione di controller.

Consideriamo una "estensione" dell'esempio precedente:

```
// file usersController.js
const usersController = {
  registerUser(req, res) {
    const { username, password } = req.body;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    // Verifica che username e password siano presenti nella req.body
    if (!username || !password) {
      res.status(400).send('Invalid username or password');
      return;
    }

    // Effettua la registrazione dell'utente
    const user = {
      username,
      password
    };
    console.log("database.saveUser(user)");

    // Restituisce una conferma di registrazione all'utente
    res.send(`User ${username} registered successfully`);
  },

  filterUsers(users, search) {
    if (!search) {
      return users;
    }

    return users.filter(user => {
      return user.name.toLowerCase().includes(search.toLowerCase());
    });
  },

  listUsers(req, res) {
    const users = [
      { id: 1, name: 'John Doe' },
      { id: 2, name: 'Jane Smith' },
      { id: 3, name: 'Bob Johnson' },
    ];

    const filteredUsers=
      usersController.filterUsers(users,req.query.search);

    res.render('users', { users: filteredUsers });
  },

};

module.exports = usersController;
```

Creiamo la pagina di registrazione register.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  <meta charset="UTF-8">
  <title>Register Form</title>
</head>
<body>
  <h1>Register Form</h1>
  <form action="/register" method="POST">
    <label for="username">Username:</label>
    <input type="text" name="username"><br><br>
    <label for="password">Password:</label>
    <input type="password" name="password"><br><br>
    <button type="submit">Register</button>
  </form>
</body>
</html>
```

Tale pagina dovrà essere inserita nella directory public del nostro progetto ed occorre aggiungere le seguenti informazioni nell'index.js:

```
app.use(express.static('public'));
app.post('/register', usersController.registerUser);
```

Di seguito la registrazione dell'utente nella pagina register.html:

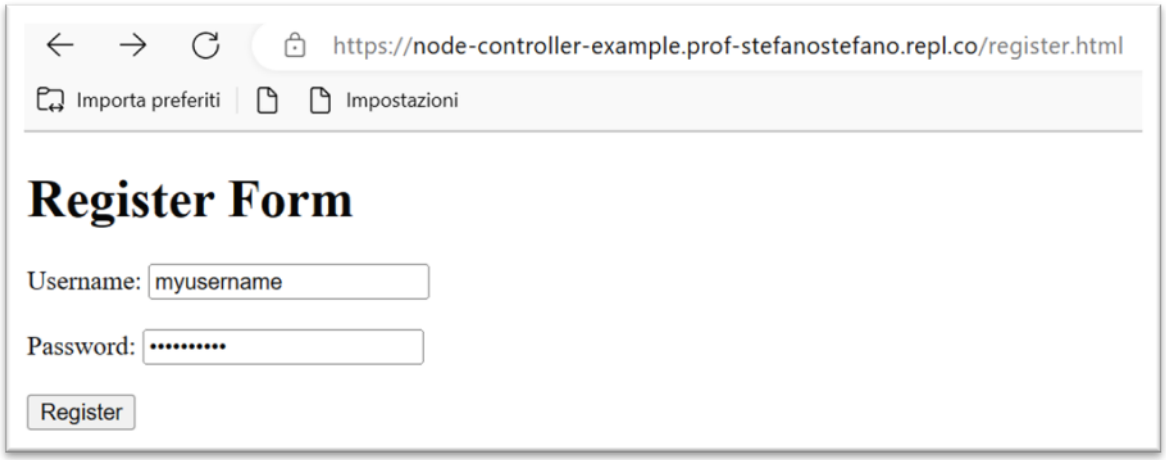


Figura 3 *- Registrazione*

L'esito è il seguente:



Figura 4 *- esito registrazione*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Model

Il model rappresenta la struttura dei dati che saranno gestiti dall'applicazione. In pratica, definisce le proprietà e i metodi che permettono di manipolare tali dati.

Ad esempio, in una app di gestione di utenti, il model potrebbe definire la struttura degli oggetti "user" che saranno salvati in un database (ad esempio un id, un username, una password, una data di creazione, ecc.), e i metodi per creare, leggere, aggiornare e cancellare questi oggetti dal database.

In Node.js esistono diverse librerie per gestire i database, tra cui Mongoose per MongoDB, Sequelize per SQL, Firebase Realtime Database per database real-time e molte altre. La scelta della libreria dipende dal tipo di database che si vuole utilizzare e dalle esigenze specifiche dell'applicazione.

Costruiamo un esempio in cui il database degli utenti sia in realtà un file users.json incluso all'interno della cartella data/:

```
[
  {
    "id": 1,
    "name": "john",
    "nick": "johnny"
  },
  {
    "id": 2,
    "name": "jane",
    "nick": "jany"
  },
  {
    "id": 3,
    "name": "bob",
    "nick": "bobby"
  }
]
```

All'interno del nostro index.js costruiamo una route che restituisca tale file json:

```
app.get('/api/users', (req, res) => {
  const users = require('./data/users.json');
  res.json(users);
});
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Nel controller usersController gestiamo la richiesta di acquisizione degli utenti dal database mediante un metodo usersFromDB:

```
const database = require('../data/database');

// file usersController.js
const usersController = {
  registerUser(req, res) {
     […]
  },

  filterUsers(users, search) {
     […]
  },

  listUsers(req, res) {
     […]
  },

  async usersFromDB(req, res) {
    const users = await database.getUsers();
    console.log(users);
    res.render('users', { users: users });
  },
};

module.exports = usersController;
```

Resta da definire il modulo database.js che inseriamo nella cartella data/

```
async function getUsers() {
  const response=await fetch('http://localhost:3000/api/users');
  const data = await response.json();
  return data;
}

async function saveUser(user) {
  // implementazione per salvare l'utente su un endpoint
  console.log("SAVEUSER");
}

module.exports = {
  getUsers,
  saveUser,
};
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Inseriamo a questo punto nell'index.js l'endPoint:

```
app.get('/dbusers', usersController.usersFromDB);
```

ATTENZIONE: la fetch usata in getUsers() usa esplicitamente async / await e non la promise con then; infatti, se si usasse la promise con con then si potrebbero avere dei problemi di "sincronia" tra chiamata e risposta, che invece mediante una gestione async / await.

Quando si esegue la richiesta all'endPoint:

```
app.get('/dbusers', usersController.usersFromDB);
```

Il controller usersController eseguirà il metodo usersFromDB il quale eseguirà una chiamata asincrona alla getUsers() del modulo database:

```
const users = await database.getUsers();
```

A questo punto il modulo eseguirà una chiamata asincrona all'API api/users che fornirà la lista degli utenti in formato json:

```
const response = await fetch('http://localhost:3000/api/users');
  const data = await response.json();
  return data;
```

A questo punto, non appena il dato sarà disponibile la pagina sarà visualizzata:

```
res.render('users', { users: users });
```

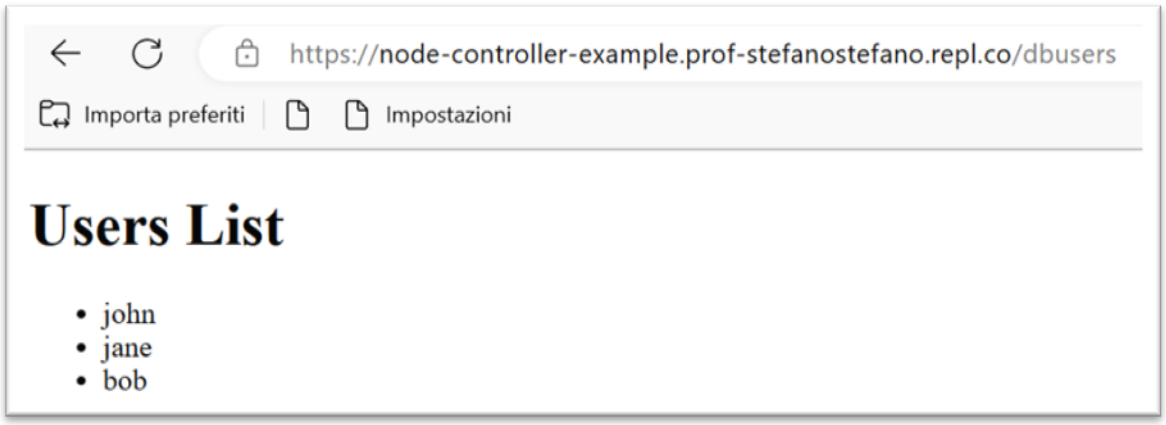


Figura 5 *- dbusers*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).